Functions in Python

1. **Method :** A Method is a function that is defined inside a class and works with objects.
    - A method is also called a member function because it is a part of a class.
2. **Function :** A Function is a block of code which only runs when it is called.
    - You can pass data, known as parameters, into a function.
    - A function can return data as a result.
    - A function helps to avoiding the code repetition.

Creating a Function

In python a function is defined using the "def" keyword, followed by a function name and parameters, and a block of code:

Example

```
def greet_f():
    print("hello every one")      #greet_f – function name
```

this create a function named greet that prints " hello every one" when called.

Note: the code inside the function must be indented. Python uses indentation to define code blocks.

Calling a Function

To call a function, use the function name and followed by parenthesis:

Example

```
def greet_f():
    print("hello every one")
greet_f()
```

Note: if you declare a function in python, you can call the same function multiple times and to avoid rewriting the same code and to improve reusability.

Function Names in python

Function names follow the same rules as variable names in python:

- A function name must start with a letter (a-z, A-Z) or underscore(_)
- A function name can contain letters, numbers, and underscroes
- Function names are case-sensitive (greet_f and greet_F are different)

Example : valid function names like

- calculate_sum()
- _private_functio()
- greet_1()

its good practice to use descriptive names that explain what the function does.

Why use functions in python?

Function are used to organize and reuse code efficiently. Instead of writing the same code again and again, you can write it once inside a function and call it whenever needed.

Main reasons

- code reusability
- reduces code duplication
- improves readability
- easy debugging
- better organization

Return Values

The return statement is used to send the results of a function back so it can be stored, reused, or used in further operations.

- A return value is the output of a function that can be stored and used later.
- When a function reaches a return statement, it stops executing and sends the result back.

Example:

Afunction that returns a value:

```
def add(a,b):
    return a+b
result = add(2,3)
print(result)
```

output: 5 (You can use returned value directly by giving some values)

without Return (for comparison)

```
def add(a,b):
    print(a+b)
result = add(2,3)
print(result)
```

output: 5

None (if a function doesn't have a return statement, it returns none by default.

Parameters

Parameters are inputs defined in a function to accept values.

Or

parameters are the variables listed in the function definition. They act as placeholders to receive values when the function is called.

Example

```
def my_function(name):
    print("hi"+name)
my_function("ram")
my_function("priya")
```

output: hiram

hipriya

Types of parameters:

1. **Default Parameter Value :** A default parameter is a parameter that has a predefined value. If no value is given during the function call, it uses the default value.
   Example :

```
def greet_1(age="21"):
    print("my age is",age)
greet_1()
greet_1(20)
```

output: my age is 21 #greet_1() is the default value
my age is 20

2. **Keyword Parameters :** A keyword parameter is when arguments are passed using parameter names, so order does not matter.

Example :

```
def student(name,age1):
    print(name,age1)
student(age1=22,name="priya")
```

output: priya 22 (even if order changes ,output is correct)

3. **Positional Parameters :** A positional parameter is when values are passed based on their position/order in the function.
   Example:

```
def add1(c,d):
    print(c+d)
add1(4,6)
```

output: 10 # 2 goes to c, 3 goes to d

4. **Arbitrary parameters (*args) :** *args allows a function to accepts **any numbers of positional arguments**. These values are stored in a **tuple**. (you can send many values, and python collects them into one group.)
   Example:

```
def total(*numbers):
    print(numbers)
total(1,2,3,4)
```

output : (1, 2, 3, 4)

5. **Keyword Arbitrary Parameters (**kwargs) :** **kwargs allows a function to accept **any number of keyword arguments**. These are stored in a **dictionary**. (you can send named values, and python stores them as key-value pairs.)
   Example :

```
def details(**info):
    print(info)
details(name="ram",age=21)
```

output : {'name': 'ram', 'age': 21}

Easy Trick to remember:

1. Default - fixed value
2. Keyword - name matters
3. Position - order matters
4. *args - many values (Tuple)
5. *kwargs - many named values (Dicitionary)

Passing a List as a Parameter : you can send any data types of parameter to function(string, number, list, dictionary ect.), and it will be treated as the same data type inside the function.

Example : if you send a list as a parameter, it will still be a list when it reaches the function.

```
def my_fun(food):
    for x in food:
        print(x)
fruits = ["apple","banana","mango"]
my_fun(fruits)
```

output: apple

banana

mango

Pass Statement : The pass statement is used as a placeholder in python that does nothing but allows empty blocks without errors.

Example-1: In a function

```
def m_func():
    pass
```

The function is empty, but no error occurs

Example-2 : In a loop

```
for i in range(5):
    pass
```

loop runs, but nothing happens

Example-3 : In condition

```
x = 10
if x > 5:
    pass
else:
    print("less than 5")
```

when the condition is true, pass runs and nothing happens.so no output is printed.

Python Conditions and If Statements

Condition : A condition is an expression that returns True or False. (it is used to make decisions in program)

- Python supports the usual logic conditions from mathematics:
  1. Equals : a == b
  2. Not Equals : a != b
  3. Less than : a < b
  4. Less than or equal to : a <= b
  5. Greater than : a > b
  6. Greater than or equal to : a >= b

  These conditions can be used in several ways, most commonly in “if statements” and “loops”.

1. **If Statement :** If statement is the most simple decision-making statement. If condition evaluates to True, the block of code inside the if Statement is executed.

   Syntax :

   if condition :

   # code runs if condition is True

   Example :

```
x = 10
if x>5:
    print("x is greater than 5")
```

   output : x is greater than 5

2. **If–else :** if -else statement is a control statement that helps in decision making based on specific conditions. When the if condition is false. The else block will be executed.

   Syntax :

   if condition1 :

   # runs if True

   else :

   # runs if False

   Example :

```
x = 5
if x>10:
    print("x is greater than 10")
else:
    print("x is less than 10")
```

   output : x is less than 10

3. **If-elif–else :** it is used to test multiple conditions and execute one block of code based on which condition is true.

Syntax:

```
if condition1 :
        # code block 1
elif condition2 :
        # code block 2

else :

        # default code block
```

Example :

```
marks = 75

if marks>=90:
    print("grade A")
elif marks>=70:
    print("grade B")
else:
    print("grade c")
```

output : grade B

4. **Nested if :** A nested if means an if inside another if is called nested if.it is used when we need to check multiple conditions step by step.

Syntax :

```
If condition1 :
        If condition2:
                # code
```

Example :

```
age = 20
id1 = True
if age>=18:
    if id1:
        print("you can enter")
```

output : you can enter

first condition is checked-then inner condition is checked.

Loop in Python

Loop : A loop is a programming structure that repeats a set of instructions until a condition becomes false or for a fixed number of times.

Why we use loops?

- To avoid writing the same code again and again.
- To work with large data easily.
- To automate repetitive tasks

Types of Loops in Python

1. **for Loop :** The for loop is used to repeat a block of code for a fixed number of times or over a collection of items.

   Syntax :

   For variable in sequence:
   #code block

   Example :

```
for i in range(1,6):
    print(i)
```

   output : 1 2 3 4 5

2. **while loop :** A while loop executes a block of code repeatedly until the given condition becomes false.

   Syntax :

   while condition:
   # code block

   Example :

```
while i<=5:
    print(i)
    i+=1
```

   output : 1 2 3 4 5

   - if the condition never becomes false, it creates an infinite loop
     eg : while True:
     print(“Hi”) #runs forever

Control Statements in Loops

Control statement : control statements are used inside loops to control the execution of the loop based on conditions.

1. **break statement :** the break statement is used to exit the loop immediately, even if the loop condition is still True.

   Syntax

   break

   Example :

```
for i in range(1,6):
    if i == 3 :
        break
    print(i)
```

   output : 1 2 #loop stops when i=3

2. **continue statement :** The continue statement is used to skip the current iteration and move to the next iteration of the loop.

   Syntax :

   Continue

   Example :

```
for i in range(1,6):

    if i == 3 :
        continue
    print(i)
```

   output : 1 2 4 5 #skips when i=3